

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

1.4

CREATE YOUR FIRST ENDPOINT

 Copy as Markdown

Create your first endpoint

You'll build a recipe generator: a Serverpod endpoint that takes a list of ingredients, asks Google's Gemini API for a recipe, and returns it to your Flutter app. Along the way you'll see that calling your server is as simple as calling a local method.

Prefer to have an AI agent build an app for you? Follow the [Quickstart](#) instead. This guide takes the hands-on path: you'll build the recipe app yourself, so you understand each piece.

INFO

The server is the right place for work you can't or shouldn't do in the Flutter app, such as calling an API secured by a secret key, accessing a database, or sending push notifications and emails. Here, it keeps your Gemini API key off the client.

Before you start

- [Serverpod installed](#). Run `serverpod version` to confirm it works.
- A free Gemini API key. On [Google AI Studio](#), sign in with a Google account and click **Create API key**.

Create the project

Use `serverpod create` to generate a new project with a server, a client, and a Flutter app:

```
$ serverpod create magic_recipe
```

The command is interactive. Step through the prompts, accepting the defaults.



Open the project's **root** folder (`magic_recipe`) in your editor, not one of the sub-packages. This keeps the analyzer in sync when code is generated and makes it easy to move between the server and app.

Add your Gemini API key

Gemini is Google's generative AI model. Your server sends it the ingredients and gets a recipe back, and the API key authenticates those calls.

Add your key to `config/passwords.yaml` in the server package. Git ignores this file, so your key stays out of version control.

```
# magic_recipe_server/config/passwords.yaml
development:
  geminiApiKey: '--- Your Gemini Api Key ---'
```

Then add the Dartantic AI package to the server. It provides a single interface for talking to AI providers, including Gemini:

```
$ cd magic_recipe_server
$ dart pub add dartantic_ai
```

Start the app

From the project's root folder, start everything with one command:

```
$ serverpod start
```

`serverpod start` generates your code, starts the server with its built-in PostgreSQL database (no Docker required), and opens the Flutter app in your default browser. The app that opens is the default Serverpod starter: enter your name and click the send button (or press Enter), and the server responds with a greeting.

Leave `serverpod start` running. It watches your project, so every time you save a file it regenerates the necessary code and hot reloads the app. You'll rely on this for the rest of the guide instead of restarting anything by hand.

Add an endpoint

Server endpoints live in `lib/src/<feature>/`, like the `greetings` endpoint the template generated. Create a file at `magic_recipe_server/lib/src/recipes/recipe_endpoint.dart`:

```
import 'package:dartantic_ai/dartantic_ai.dart';
import 'package:serverpod/serverpod.dart';
```

```

/// This is the endpoint that will be used to generate a recipe using the
/// Google Gemini API. It extends the Endpoint class and implements the
/// generateRecipe method.
class RecipeEndpoint extends Endpoint {
  /// Pass in a string containing the ingredients and get a recipe back.
  Future<String> generateRecipe(Session session, String ingredients) async {
    final geminiApiKey = session.passwords['geminiApiKey'];
    if (geminiApiKey == null) {
      throw Exception('Gemini API key not found');
    }

    // Configure the Dartastic AI agent for Gemini before sending the prompt.
    final agent = Agent.forProvider(
      GoogleProvider(apiKey: geminiApiKey),
      chatModelName: 'gemini-3.5-flash-lite',
    );

    final prompt =
      'Generate a recipe using the following ingredients: $ingredients. '
      'Always put the title of the recipe in the first line, followed by the '
      'instructions. The recipe should be easy to follow and include all '
      'necessary steps.';

    final response = await agent.send(prompt);

    final responseText = response.output;

    if (responseText.isEmpty) {
      throw Exception('No response from Gemini API');
    }

    return responseText;
  }
}

```

The endpoint reads your Gemini key from `session.passwords`, which Serverpod populates from the `passwords.yaml` file you edited earlier.

INFO

Endpoint methods take a `Session` as their first parameter and return a typed `Future` or `Stream`. You can pass and return primitive types or any model defined in a `.spy.yaml` file. The class name's `Endpoint` suffix is dropped on the client, so `RecipeEndpoint` is called via `client.recipe`. See [How it works](#) for how that call reaches the server.

Save the file. Because `serverpod start` is watching, it regenerates the client bindings for `generateRecipe` automatically. You'll see it run in the terminal.

Call it from your app

Your app's UI lives in `magic_recipe_flutter/lib/screens/`, where the template already added a `GreetingsScreen`. Add a recipe screen alongside it.

Create `magic_recipe_flutter/lib/screens/recipe_screen.dart`:

```
import 'package:flutter/material.dart';

import '../client.dart';
import 'greetings_screen.dart';

class RecipeScreen extends StatefulWidget {
  const RecipeScreen({super.key});

  @override
  State<RecipeScreen> createState() => _RecipeScreenState();
}

class _RecipeScreenState extends State<RecipeScreen> {
  /// Holds the last result, or null if there's no result yet.
  String? _resultMessage;

  /// Holds the last error message, or null if there's no error yet.
  String? _errorMessage;

  final _textEditingController = TextEditingController();

  bool _loading = false;

  void _callGenerateRecipe() async {
    try {
      setState(() {
        _errorMessage = null;
        _resultMessage = null;
        _loading = true;
      });

      final result = await client.recipe.generateRecipe(
        _textEditingController.text,
      );

      setState(() {
        _resultMessage = result;
        _loading = false;
      });
    } catch (e) {
      setState(() {
        _errorMessage = '$e';
        _loading = false;
      });
    }
  }

  @override
  Widget build(BuildContext context) {
    return Padding(
```

```
padding: const EdgeInsets.all(16),
child: Column(
  children: [
    TextField(
      controller: _textEditingController,
      decoration: const InputDecoration(
        hintText: 'Enter your ingredients',
      ),
    ),
    const SizedBox(height: 16),
    ElevatedButton(
      onPressed: _loading ? null : _callGenerateRecipe,
      child: _loading
        ? const Text('Loading...')
        : const Text('Generate Recipe'),
    ),
    const SizedBox(height: 16),
    Expanded(
      child: SingleChildScrollView(
        child: ResultDisplay(
          resultMessage: _resultMessage,
          errorMessage: _errorMessage,
        ),
      ),
    ),
  ],
),
);
}
```

`client` comes from `client.dart`, where the template already wired it to talk to your server, and `ResultDisplay` is reused from `greetings_screen.dart`.

Now show the recipe screen instead of the greeting demo. In `magic_recipe_flutter/lib/main.dart`, add the import:

```
import 'screens/recipe_screen.dart';
```

Then, in the `MyHomePage` widget, change the body from `GreetingsScreen` to `RecipeScreen`:

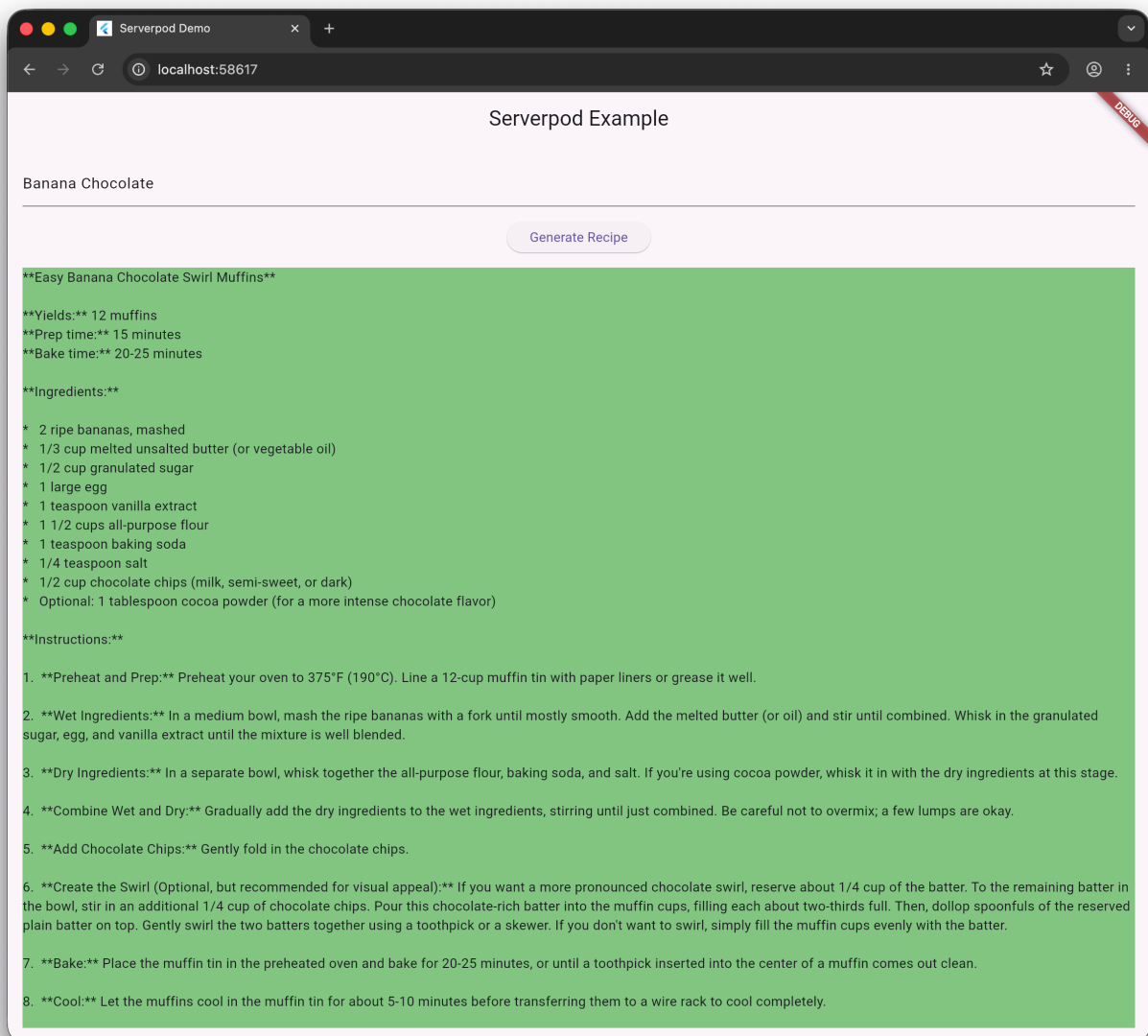
```
body: const RecipeScreen(),
```

Save. UI edits like this would normally hot reload, but adding the endpoint also changed the generated client. The app's `client` is created once by `initializeClient()` in `main()`, which only re-runs on a restart, so the app needs a hot restart to pick up the new `client.recipe` endpoint.

In the `serverpod start` terminal:

- Press **R** to hot restart.

Then enter some ingredients and tap **Generate Recipe**. The app calls your endpoint and displays the result:



Next steps

You've created an endpoint and called it from your app, passing a string back and forth. Next, you'll return structured data using a Serverpod model, with serialization handled for you. Leave `serverpod start` running; you'll keep building on the same app.